Cold Spray for Grins

Ruiyu Ou

August 17, 2026

Contents

1 Introduction

2 Gen AI Usage Disclosure

All physics and coding is done by myself. Gen AI platforms such as Claude or ChatGPT are used to expedite reference searching, and deal with debugging. This LaTeX code still does not work properly despite Gen AI working on it. Maybe because I am using the free versions so they can't handle my disgustingly poor LaTeX code (may be more due to how I structured the dirs).

3 Free Boundary Problem

3.1 The Stefan Problem

3.1.1 Description and Connection with Cold Spray

The Stefan problem describes heat transfer in a material containing a moving phase boundary, where the interface between two phases evolves over time as latent heat is absorbed or released. A classic example is the melting or freezing of ice submerged in a body of water, where the solid–liquid interface moves in response to heat conduction.

A particularly important engineering application of Stefan-type problems is welding. In conventional welding processes, metal at the joint interface is heated above its melting temperature, with or without the addition of a filler material, before solidifying to form a strong metallurgical bond. Cold spray deposition shares several characteristics with explosive welding, although the underlying mechanisms differ. During cold spray, metallic particles impact the substrate at supersonic velocities, producing severe plastic deformation and localized adiabatic heating. These conditions promote adiabatic shear instability and oxide-layer disruption, enabling solid-state metallurgical bonding between the particles and the substrate without bulk melting.[**?**]

3.1.2 The Mathematical Problem

We will consider the 2 phase Stefan problem in 2 dimensions. [**?**]

We denote with Ω_i as the region for each phase i, $T_i(\vec{x}, t)$ as the temperature in the respective region, $\partial\Omega_i$ as the boundaries, α_i as the thermal diffusivity, and Q_i as sources of heat.

Within each region, it will satisfy the inhomogeneous heat diffusion equation with their respective thermal diffusivity and Robin boundary conditions. One specific condition is that the temperature at the free boundary be equivalent to the critical phase transition temperature.

$$\frac{\partial T_i}{\partial t} = \nabla \cdot (\alpha_i \nabla T_i) + Q_i \text{ ; for } \vec{x} \in \Omega_i \tag{1}$$

$$a(x)T_i(x) + b(x)\partial_n T_i(x) = g(x) \text{ ; for } \vec{x} \in \partial\Omega_i \tag{2}$$

The interface transitions between each phases i and j with a free surface velocity $\vec{V}$ that satisfies the following differential equation.

$$\vec{V_i}(x,t) = \frac{\mathrm{d}x}{\mathrm{d}t} \text{ ; for } \vec{x} \in \partial\Omega_i \tag{3}$$

$$L_{ij}\vec{V_i} = \alpha_i \partial_{n_i} T_i - \alpha_j \partial_{n_j} T_j \text{ ; for } \vec{x} \in \partial\Omega_i \cap \partial\Omega_j \tag{4}$$

With L_{ij} being the latent heat of phase transition between phases i and j, and ∂_{n_i} being the limit of the outward normal derivative of Ω_i.

3.2 FreeFEM Implementation

3.2.1 The Chosen Toy Problem

We choose the classic problem of ice melting and solidifying. CGS units are used here.

The necessary constants are stated in the table below.

Name	Notation in code	Numerical value
Thermal diffusivity of ice	k_{ice}	1.335E-2 cm^2/s
Thermal diffusivity of water	k_{water}	1.4E-3 cm^2/s
Latent heat of melting	L	334E-2 cm^2/s^2

Where we have chosen to scale the latent heat of melting by 1E-11 to amplify the effects of melting for visual purposes.

We consider the problem in 1cm by 1cm square bisected by the given parametric function.

$$x(t) = 0.3 + 0.4\frac{\frac{1}{1+\exp(-k(t-0.5))} - \frac{1}{1+\exp(0.5k)}}{\frac{1}{1+\exp(-0.5k)} - \frac{1}{1+\exp(0.5k)}} \tag{5}$$

$$y(t) = 1 - t \tag{6}$$

where the parameter k determines the smoothness of the jump. The region on the left will be ice, and the region on the right will be water. See Fig **??**

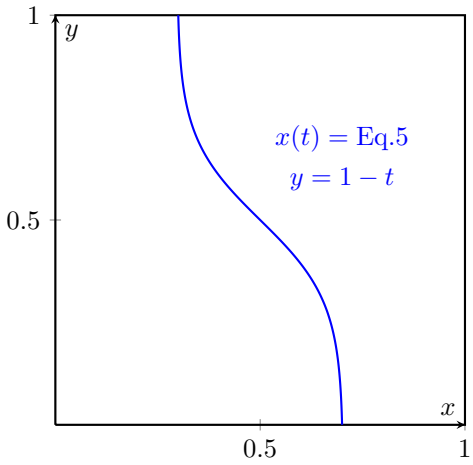


Figure 1: The curve bisecting the 2 phases. The region on the left will be ice, and the region on the right will be water.

We choose the below boundary conditions.

$$\frac{\partial T_{ice}}{\partial n} = 0 \text{ ; for (x,y) on the left, upper, and lower edges.} \tag{7}$$

$$T_{ice} = 0 \text{ ; for (x,y) on the free surface.} \tag{8}$$

$$\frac{\partial T_{water}}{\partial n} = 0 \text{ ; for (x,y) on the upper and lower edges.} \tag{9}$$

$$T_{water} = 0 \text{ ; for (x,y) on the free surface.} \tag{10}$$

$$T_{water} = 5 \text{ ; for (x,y) on the right edge.} \tag{11}$$

The initial temperatures will be -1 for ice, and 1 for water everywhere. Finally, with no external heat source – $Q = 0$.

3.2.2 The Numerical Implementation

We build the meshes separately to obtain the normal vectors at the boundaries, and assign the appropriate regions as ice and water by use of an indicator function "regIce".

```
// Left region (ice)
mesh ThL = buildmesh(S1(n) + S2(n) + S3(n) + S4(n) + D1(n));

// Isolate left region (ice): region containing point (0.1,
    0.5)
int regIce = ThL(0.1, 0.5).region;
mesh ThIce   = trunc(ThL, region == regIce);
mesh ThWater = trunc(ThL, region != regIce);
```

```
mesh IceStart= ThIce;
```

We then build the piecewise quadratic finite element space.

```
fespace ThI(ThIce, P2);
ThI dxI,dyI, i1, i2; // Mesh moving fespace
ThI iu, iuold, iv; // temperature fespace

fespace ThW(ThWater, P2);
ThW dxW,dyW, w1, w2;
ThW wu, wuold, wv;
```

Where we keep the old temperature to use the Crank-Nicholson solver.

We then solve the homogeneous heat diffusion problem with Crank-Nicholson.

```
    real theta = 0.5; // Crank-Nicholson

problem HeatICN(iu, iv)
    = int2d(ThIce)(
          iu*iv/dt
        + theta * kice * (
              dx(iu)*dx(iv)
            + dy(iu)*dy(iv)
        )
    )
    - int2d(ThIce)(
          iuold*iv/dt
        - (1-theta) * kice *(
              dx(iuold)*dx(iv)
            + dy(iuold)*dy(iv)
        )
    )
    + on(diag,  iu=0)
    ;
problem HeatWCN(wu, wv)
    = int2d(ThWater)(
          wu*wv/dt
        + theta * kwater * (
              dx(wu)*dx(wv)
            + dy(wu)*dy(wv)
        )
    )
    - int2d(ThWater)(
          wuold*wv/dt
        - (1-theta) * kwater *(
              dx(wuold)*dx(wv)
            + dy(wuold)*dy(wv)
        )
    )
    + on(right, wu=5)
    + on(diag,  wu=0)
```

```
    ;
```

We also set up a Laplace problem for moving the boundary, which will be elaborated upon in the next subsubsection.

```
    problem MoveIX(dxI, i1) =
    int2d(ThIce)(dx(dxI)*dx(i1) + dy(dxI)*dy(i1))
  + on(left, dxI = 0)
  + on(diag,  dxI = dt/L*(kice*dx(iu)*N.x-kwater*dx(wu)*N.x)
     )
  ;

problem MoveIY(dyI, i2) =
    int2d(ThIce)(dx(dyI)*dx(i2) + dy(dyI)*dy(i2))
  + on(diag,  dyI = dt/L*(kice*dy(iu)*N.y-kwater*dy(wu)*N.y)
     )
  + on(top, dyI = 0)
  + on(bottom, dyI = 0)
  ;

problem MoveWX(dxW, w1) =
    int2d(ThWater)(dx(dxW)*dx(w1) + dy(dxW)*dy(w1))
  + on(right, dxW = 0)
  + on(diag,  dxW = -dxI)
  ;

problem MoveWY(dyW, w2) =
    int2d(ThWater)(dx(dyW)*dx(w2) + dy(dyW)*dy(w2))
  + on(diag,  dyW = -dyI)
  + on(top, dyW = 0)
  + on(bottom, dyW = 0)
  ;
```

3.2.3 Challenge with Moving the Boundary

We quickly ran into a challenge with moving the boundary.

The numerical formulation for the boundary move increment Δx is given by the following equation.

$$\Delta x_i = \Delta t \cdot \vec{V} = \frac{\Delta t}{L} \left(\alpha_i \partial_{n_i} T_i - \alpha_j \partial_{n_j} T_j \right) \tag{12}$$

The first numerical formulation on incrementing the boundary only moved the finite element points on the boundary. This causes some of the finite elements to flip and FreeFEM terminates with an error of negative area.

To resolve the error, one has a few choices. The simplest way forward is to adjust the time step such that the boundary does not move enough to flip the finite elements, and then remesh to eliminate bad mesh geometry at the boundary. However, this approach severely restricts the time discretization stepsize

by the size of the finite elements, and is difficult to predict the appropriate step size.

Naturally then, the choice will be to move all points, and remesh whenever necessary. The problem now is to interpolate the increment everywhere within the domain given the boundary increment. Thus, we choose to solve the following Laplace problem.

$$\nabla^2 \vec{X}_i = 0 \quad \text{in } \Omega_i \tag{13}$$

$$\vec{X}_i = \Delta x_i \quad \text{on } \partial\Omega_i \cap \partial\Omega_j \tag{14}$$

Where $\vec{X}_i$ will be the mesh increment everywhere in Ω_i. Given all the regularization theorems regarding the Laplace operator, one can guess that this allows a much wider range of time step sizes to be used. We present no analysis on the validity of this guess here.

3.2.4 Results

The results can be visualized as a time series in Paraview, or any .vtu viewer.

The ice region grows in size as it solidifies due to a higher thermal diffusivity, as it loses heat energy into the water region. It then stops growing and begins to melt as the water which is maintained at 5 degree celsius on the right bound inputs heat into the ice region.

In the end, we get a net loss of ice region.

4 Contact Mechanics

4.1 The Contact Problem

4.1.1 Description and Connection with Cold Spray

Contact problems describe the mechanical interaction between two or more deformable bodies that come into contact. The primary challenge is determining the contact interface while enforcing the non-penetration constraint. Depending on the physical model, frictional effects may also be incorporated to describe tangential resistance and sliding along the contact surface.

In cold spray additive deposition, metallic particles impact a target substrate at supersonic velocities, producing severe plastic deformation and localized adiabatic heating. These conditions promote adiabatic shear instability amongst other effects, which allows the particle to then metallurgically bond to the substrate material. [**?**] Particle-to-subtrate and particle-to-particle bonding are both present in cold spray deposition, which may be of different materials. Determination for the correct physics locally necessarily requires the contact problem.

4.1.2 The Mathematical Problem

Entire textbooks are devoted to contact mechanics; therefore, only a brief overview is presented here. [**?**]

Contact mechanics studies the interaction between two bodies when they come into physical contact. The primary objective is to determine the contact forces and resulting deformations while enforcing the appropriate contact constraints.

For two bodies with an interface Γ, the fundamental unilateral contact conditions are the Signorini conditions:

$$g_n \geq 0, \qquad t_n \leq 0, \qquad g_n t_n = 0, \tag{15}$$

where g_n is the normal gap and t_n is the normal contact traction. These conditions state that the bodies cannot penetrate, contact forces can only be compressive, and a contact force exists only when the bodies are touching.

For frictional contact, the tangential traction is additionally constrained by Coulomb's law,

$$\|\mathbf{t}_t\| \leq \mu |t_n|, \tag{16}$$

with equality during sliding, where μ is the friction coefficient.

4.2 Elastodynamics

The cold spray process is necessarily a dynamic one, so one must resolve the physics incrementally.

4.2.1 Linear Elasticity

One may model the cold spray process as an elastic-plastic problem, though the elasticity portion is far less significant.

The strong form of linear elasticity is given by Cauchy's equations.

$$\rho \ddot{x} = \nabla \cdot \sigma + f \tag{17}$$

$$\sigma = \lambda \operatorname{tr}(\varepsilon)\mathbf{I} + 2\mu\epsilon \tag{18}$$

$$\varepsilon = \frac{1}{2}(\nabla x + (\nabla x)^T) \tag{19}$$

We will now convert this into the weak form that will be more useful for us which will be elaborated upon in the next subsection. v will be our test functions.

$$\int \rho \ddot{x} \cdot v = \int (\nabla \cdot \sigma) \cdot v + \int f \cdot v \tag{20}$$

$$\int \rho \ddot{x} \cdot v + \int \sigma(x) : \varepsilon(v) = \int f \cdot v \tag{21}$$

By application of the divergence theorem on the internal stress term.

We expand the stress and strain tensor with the constitutive equations **??** and **??**.

$$\int \rho\ddot{x} \cdot v + \int \lambda(\boldsymbol{\nabla} \cdot x)(\boldsymbol{\nabla} \cdot v) + 2\mu\epsilon(x) : \epsilon(v) = \int f \cdot v \tag{22}$$

Naturally, we convert the equation into Galerkin's formulation to then input into FreeFEM. Einstein summation convention here will be used.

$$x(t) = X_i(t)u_i(x) \tag{23}$$

$$\rho\ddot{X}_i \int u_i \cdot v_j + X_i \int \lambda(\boldsymbol{\nabla} \cdot u_i)(\boldsymbol{\nabla} \cdot v_j) + 2\mu\varepsilon(u_i) : \varepsilon(v_j) = \int f \cdot v_j \tag{24}$$

Observe that this gives us the following equation.

$$M_{ij}\ddot{x}_j + K_{ij}x_j = f_i \tag{25}$$

$$M_{ij} = \int u_j \cdot v_i \ ; \ K_{ij} = \int \lambda(\boldsymbol{\nabla} \cdot u_j)(\boldsymbol{\nabla} \cdot v_i) + 2\mu\varepsilon(u_j) : \varepsilon(v_i) \ ; \ f_i = \int f \cdot v_i \tag{26}$$

Which is the form to then plug into the α integrators such as HHT-α and Newark-β.

4.2.2 Newark Beta

Here we will present the incremental Newark Beta integrator for structural dynamics.[**?**]

We follow Gavin, H.'s lecture notes on incremental formulation of the Newark integrator.

The Newark integrator begins with the following assumptions on the form of the differential equation and the discretizations.

$$M\ddot{x} + C\dot{x} + Kx + R(x, \dot{x}) = f \tag{27}$$

$$\dot{x}_{i+1} \approx \dot{x}_i + h\left[(1-\gamma)\ddot{x}_i + \gamma\ddot{x}_{i+1}\right] \tag{28}$$

$$x_{i+1} \approx x_i + h\dot{x}_i + h^2\left[\left(\frac{1}{2} - \beta\right)\ddot{x}_i + \beta\ddot{x}_{i+1}\right] \tag{29}$$

Where M is the mass matrix, C the dissipation matrix, K the stiffness matrix, R will be any nonlinear components, f being any external forces, and A_i being $A(t = ih)$.

The free parameters β and γ are left to be tuned for desired integrator behaviors. Choices such as $\gamma < 1/2$, the integrator is conditionally stable, whilst the chosen method for this project is implicit and unconditionally stable with $\beta = 1/4$ and $\gamma = 1/2$.

We now solve for $\ddot{x}_{i+1}$.

$$\ddot{x}_{i+1} = \frac{1}{\beta h^2}(x_{i+1} - x_i) - \frac{1}{\beta h}\dot{x}_i - \left(\frac{1}{2\beta} - 1\right)\ddot{x}_i \tag{30}$$

Giving us this formula for the next velocity.

$$\dot{x}_{i+1} = \frac{\gamma}{\beta h}(x_{i+1} - x_i) - \left(\frac{\gamma}{\beta} - 1\right)\dot{x}_i + h\left(1 - \frac{\gamma}{2\beta}\right)\ddot{x}_i \tag{31}$$

4.2.3 Numerical FreeFEM Implementation

We consider a linear elastic block resting on a table, slumping under the effect of gravity. The constant parameters are as follows.

Name	Notation in Code	Value
Gravity	g	-9.8 m/s^2
Density	ρ	1
Young's modulus	E	1E3 $kg/m/s^2$
Poisson modulus	nu	0.3
Newark parameter	beta	0.25
Newark parameter	gamma	0.5
Rayleigh dissipation parameter	AlphaM	0.01
Rayleigh dissipation parameter	AlphaK	0.01

We introduce some notation to simplify the equations **??** and **??**.

$$c_0 = \frac{1}{\beta h^2}\ ;\ c_1 = \frac{1}{\beta h}\ ;\ c_2 = \frac{1}{2\beta} - 1 \tag{32}$$

$$c_3 = \frac{\gamma}{\beta h}\ ;\ c_4 = \frac{\gamma}{\beta} - 1\ ;\ c_5 = h\left(1 - \frac{\gamma}{2\beta}\right) \tag{33}$$

As shown in the code below.

```
// Beta Newmark constants
real c0 = 1./(beta*dt*dt);
real c1 = 1./(beta*dt);
real c2 = 1./(2.*beta) - 1.;
real c3 = gamma/(beta*dt);
real c4 = gamma/beta - 1.;
real c5 = dt*(gamma/(2*beta) - 1.);
```

The Lame parameters for the stress tensor are given as follows.

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}\ ;\ \mu = (3 * 10^{-2})\frac{E}{2(1+\nu)} \tag{34}$$

Where we have chosen to scale the 2nd Lame parameter by 3e-2 to amplify visual effects by making the block jigglier.

A particular sanity check is that under dampening, the system will converge to the steady state, which one can trivially compute. So, we add in the Rayleigh dissipation terms into the dissipation matrix C.

$$C = \alpha_M M + \alpha_K K \tag{35}$$

Here we introduce some more notations to simplify the Galerkin formulation.

$$M(a) = \int a \cdot v \ ; \ K(a) = \int \lambda(\boldsymbol{\nabla} \cdot a)(\boldsymbol{\nabla} \cdot v) + 2\mu\varepsilon(a) : \varepsilon(v) \tag{36}$$

As shown in the code below.

```
// Macros M and K as described in the PDF
macro M(a) (a[0]*pvi+a[1]*pvj) //
macro K(a) (lambda*(div(a)*div([pvi,pvj]))+2*muK*mu*eps(a)'*
    eps([pvi,pvj])) //
```

Altogether, we form the Galerkin formulation.

$$\begin{aligned}
M\ddot{x}_{i+1} + C\dot{x}_{i+1} + Kx_{i+1} - f_{i+1} = 0 \\
M(c_0(x_{i+1} - x_i) - c_1\dot{x}_i - c_2\ddot{x}_i) + \\
(\alpha_M M + \alpha_K K)(c_3(x_{i+1} - x_i) - c_4\dot{x}_i - c_5\ddot{x}_i) + \\
Kx_{i+1} - f_{i+1} = 0 \\
c_0M(x_{i+1}) - c_0M(x_i) - c_1M(\dot{x}_i) - c_2M(\ddot{x}_i) + \\
\alpha_M(c_3M(x_{i+1}) - c_3M(x_i) - c_4M(\dot{x}_i) - c_5M(\ddot{x}_i)) + \\
\alpha_K(c_3K(x_{i+1}) - c_3K(x_i) - c_4K(\dot{x}_i) - c_5K(\ddot{x}_i)) + \\
K(x_{i+1}) - f_{i+1=0}
\end{aligned}$$

Finally, we shuffle terms to have all additions in 1 integral and all subtractions in another.

$$\begin{aligned}
c_0M(x_{i+1}) + \alpha_M c_3M(x_{i+1}) + \alpha_K c_3K(x_{i+1}) + K(x_{i+1}) \\
- \\
c_0M(x_i) + c_1M(\dot{x}_i) + c_2M(\ddot{x}_i) + \\
\alpha_M(c_3M(x_i) + c_4M(\dot{x}_i) + c_5M(\ddot{x}_i)) + \\
\alpha_K(c_3K(x_i) + c_4K(\dot{x}_i) + c_5K(\ddot{x}_i)) + \\
f_{i+1} = 0
\end{aligned}$$

As shown in the code.

```
problem BetaNewark([px,py],[pvi,pvj])
= int2d(Ph)(
        c0*M([px,py])+
                AlphaM*c3*M([px,py])+
                AlphaK*c3*K([px,py])+
                K([px,py])
        )
- int2d(Ph)(
                c0*M([un,vn])+
                c1*M([uvn,vvn])+
                c2*M([uan,van])+
```

```
                AlphaM*(c3*M([un,vn])+c4*M([uvn,vvn])+c5*M([
                    uan,van]))+
                AlphaK*(c3*K([un,vn])+c4*K([uvn,vvn])+c5*K([
                    uan,van]))+
                g*pvj
        )
+ on(bottom, py=0);
```

Using FreeFEM's builtin hTriangle function, we can obtain the size of the finite elements. This way we can adequately choose the time step size for a good CFL number. The minimum size element in this case has the size of 0.0820061m.

Wave Type	Speed	Time Step for CFL=1
Pressure	24.4949 m/s	0.00334788s
Shear	3.39683 m/s	0.0241419s

We choose the Eulerian description and do not adapt the meshes here in the toy problem, therefore the mesh sizes never changes, so we choose a time step of 0.00334788s. Due to the choice of Newark parameters, no stability issues will be present. The specific choice of time step is for better convergence.

4.2.4 Results

The results can be viewed as a time series in Paraview, or any .vtu viewer. The block jiggles and the movements slowly decay away.

4.3 FreeFEM Implementation

We adopt a recent development in contact mechanics developed by the Air Force Research Labs of Schwarz alternating method [**?**], though we deviate from the Dirichlet-Neumann method they chose, and instead opted for Robin-Robin transmission.

4.3.1 Physical Model

We choose the problem of a ball 1 meter in diameter falling under gravity and undergoing frictionless contact on top of a base plate of 5 meters wide and 1 meter thick.

The materials with be the same for both the base plate and the ball, with the following parameters.

```
// We will be assuming a density of 1kg/m^3
real E          = 1E3;  // Young's modulus: kg/m/s^2
real nu         = 0.45; // Poisson modulus: 1
```

4.3.2 Trouble with Dirichlet-Neumann

We deviated from the original paper and implemented the Robin-Robin transmission method instead. The core reason is the difficulty to implement the correct contact Dirichlet conditions in FreeFEM.

The problem is two-fold. The built-in FreeFEM on(...) function enforces Dirichlet conditions on the entire labeled boundary. Well nothing really stops me from mathematically enforcing a condition such as $a(x)F(x) = g(x)$ by multiplying an extra $a(x)$ to get something like Karush-Kuhn-Tucker. It appears FreeFEM's on(...) function is also very restrictive on the form of the equation, and that may not be possible. If there is a method to dynamically relabel boundaries to label contact surfaces, and/or have the desired Dirichlet condition be syntactically correct for FreeFEM to compile, neither myself nor GenAI can find an appropriate source where that is done. As a note to future readers/learners, I did not make much of an attempt to implement this in the FreeFEM package, it may be more readily possible in another FEA packages, or simply needs more tinkering in FreeFEM.

Noting the remarkably simpler to implement Robin conditions, as it is merely an additional boundary integral, I have chosen to swap to the Robin-Robin formulation.

4.3.3 Numerical Implementation

We present an overview of the Schwarz DDM algorithm.

```
    1. Check contact
    1. (a) if contact: Resolve contact with Robin-Robin
       Schwarz DDM
    1. (b) else: normal time integration
```

```
2. Update a contact history function
3. Recheck for contact, and restart if there is contact
```

Contact detection is done via 2 separate routes.

```
1. If in contact at last time step, check if the
   traction stress is compressive
2. If not in contact at last time step, check if there
   is domain overlap
```

If the traction is not in compression, then the 2 are likely in the rebound process and will detach soon. The 2nd condition is obvious.

We now detail the Robin-Robin transmission portion. We simplify to the 2 body problem. The n body generalization is straight forward. We denote T_i as the traction stress on body i, which is given by PN where P is the 1st Piola-Kirchoff stress, and N is the surface normal. Γ is the contact surface. The nonoverlapping Robin-Robin is then given by the following equations.

$$
\begin{cases}
M_1\ddot{u}_1^{(s+1)} + K_1 u_1^{(s+1)} = F_1 & \text{in } \Omega_1, \\
a_1(x)\partial_n u_1^{(s+1)} + b_1(x) u_1^{(s+1)} = g_1(x) & \text{on } \partial\Omega_1 \setminus \Gamma, \\
\alpha_{12} T_1^{(s+1)} + \beta_{12} u_1^{(s+1)} = \lambda_1^{(s+1)} & \text{on } \Gamma.
\end{cases}
$$

$$
\begin{cases}
M_2\ddot{u}_2^{(s+1)} + K_2 u_2^{(s+1)} = F_2 & \text{in } \Omega_2, \\
a_2(x)\partial_n u_2^{(s+1)} + b_2(x) u_2^{(s+1)} = g_2(x) & \text{on } \partial\Omega_2 \setminus \Gamma, \\
\alpha_{21} T_2^{(s+1)} + \beta_{21} u_2^{(s+1)} = \lambda_2^{(s+1)} & \text{on } \Gamma.
\end{cases}
$$

$$
\lambda_1^{(s+1)} = \theta_1 \left[\alpha_{12} T_2^{(s)} + \beta_{12} u_2^{(s)} \right] - (1-\theta_1)\lambda_1^{(s)}
$$

$$
\lambda_2^{(s+1)} = \theta_2 \left[\alpha_{21} T_1^{(s+1)} + \beta_{21} u_1^{(s+1)} \right] - (1-\theta_2)\lambda_2^{(s)}
$$

Where α_{ij}, β_{ij}, and θ_i are parameters to tune for the desired behavior. Broadly speaking, θ controls the convergence rate of the Schwarz iteration, α_{ij} controls the magnitude of the repelling force, and βij is how much the contact with another domain surface repels another domain. If one changes β_{ij} to 10, one would observe that the ball is instantly repelled with massive local deformation when barely 1 or 2 nodes make contact with the base plate, impossible for the elasticity to do that in reality. Choosing the appropriate parameters is the study of optimal Schwarz methods [**?**].

One would iteratively solve the above equation with the appropriate Robin boundary condition not on the contact surface, and the prescribed Robin condition on the contact surface.

We choose $\theta_i = 1$, $\alpha_{ij} = 10^{-3}$, and $\beta_{ij} = 1$

Energy is unlikely to be conserved during contact due to the existence of high frequency noise, but one can reasonably expect lower errors when the domains detach and the noise is damped. Like most difficult numerical processes, there are sources of instability. To deal with them, one would add controlled dissipation.

We integrate in time using the Newark beta integrator, with the following parameters to damp out high frequency noise.

$$\gamma = 0.9 \ ; \ \beta = \frac{1}{4}\left(\frac{1}{2} + \gamma\right)^2$$

For further detail, please read Mota et al. 's paper [**?**].

4.3.4 Results

The results can be viewed as a time series in Paraview, or any .vtu viewer.

The ball bounces on top of the jiggling base plate.

5 Plasticity

5.1 The Elastoplasticity Problem

5.1.1 Description and Connection with Cold Spray

Plasticity is the study of materials that undergo irreversible, permanent deformations under large loads.

Cold spray deposition results from micron sized metal particles impacting a target substrate at supersonic speeds. Plastic and thermomechanical effects are the dominant physics in cold spray deposition. Under the viscoplastic deformation and thermomechanical behaviors, the metal particles undergo shear thinnign then metallurgically binds the target substrate. Understanding the plastic processes is necessary for modeling cold spray deposition.

5.1.2 The Mathematical Problem

Plasticity is inherently an experimental science. The mathematical theories of plasticity are formulated to represent experimental findings and physical models, and such is more phenomenological. The physical models of plasticity is concerned with how plasticity occurs.

The physical material of interest will completely dictate inelastic processes. The materials of interest can vary greatly from fairly regular metals, to soil types, and even rocks and concrete. Respectively, some modes of fault: slip bands, compactification of soil or voids and shearing, and opening and closing of cracks. [**?**]

To describe the onset and accumulations of permanent deformation, known as yielding, we need a criterion for the transition from elastic to plastic deformation. From physical experiments, the transition from elastic to plastic may not be obvious, if such a point even exists in reality. There are then 2 main types of constitive models for plastic deformation. One assuming the existence of a yield point, and one that does not. Here we present the former. [**?**]

Suppose there exists a function of all internal state variables called the yield function – $F(q)$. We denote the collection of all state variables with q. We assume the following.

$$\text{if } F(q) < 0, \text{ elastic deformation for all such q. Plastic otherwise.} \tag{37}$$

Naturally, the manifold given by $F(q) = 0$ represent the point of transition from elastic to plastic deformation. The manifold is known as the yield surface.

The specific $F(q)$ will depend on the model.

As all thermodynamical models go, we assume the existence of a free energy ψ, depending on the elastic strain and internal state variables. We then obtain the following equations for stress and hardening.

$$\sigma = \bar{\rho}\frac{\partial \psi}{\partial \varepsilon^e} \ ; \ A = \bar{\rho}\frac{\partial \psi}{\partial \alpha} \tag{38}$$

Where σ is the stress, ε^e is the elastic strain, A the thermomechanical hardening process, and α the internal state variables.

A complete thermomechanical description of a plastic material also needs the evolution laws for the internal state variables. We present the following equations for the time evolution of the plastic strain ε^p, and internal variables α.

$$\dot{\varepsilon}^p = \dot{\gamma} N(\sigma, A) \ ; \ \dot{\alpha} = \dot{\gamma} H(\sigma, A) \tag{39}$$

$$N = \frac{\partial f}{\partial \sigma} \ ; \ H = -\frac{\partial f}{\partial A} \tag{40}$$

Where $\dot{\gamma}$ is a proportionality parameter known the plastic multiplier, N known as the flow vector, H known as the hardening modulus, adn they are derived from a plastic flow potential f.

The evolution laws are complemented with a set of sensible physics known as the loading/unloading conditions, which may be of a familiar form from some contact problems.

$$F \leq 0 \ ; \ \dot{\gamma} \geq 0 \ ; \ \dot{\gamma} F = 0 \tag{41}$$

Recall the criterion for the onset of plastic deformation Eq **??**. If the conditions are purely elastic, no thermomechanical processes occur, and the evolution of internal state variables are null. In this case we can trivially force $\dot{\gamma} = 0$. If the conditions are elastoplastic, there are some form of thermomechanical process so $\dot{\gamma} \geq 0$ and $F \geq 0$. However, physically the time evolution of the elastoplastic body is continuous in phase space. It must continuously move from 1 internal state corresponding to elastic deformation $F < 0$ to plastic deformation $F \geq 0$. With a trivial connectedness consideration, it must begin plastically deforming at $F = 0$, and the internal state variables will evolve with it as the current stress is equivalent to the yield stress. Which then naturally gives us that $\dot{\gamma} \geq 0$, and $F = 0$ under plastic deformation.

All together, we have these equations that describe plasticity.

$$\text{Additivity of strain: } \varepsilon = \varepsilon^e + \varepsilon^p \tag{42}$$

$$\text{Free energy: } \psi(\epsilon^e, \alpha) \tag{43}$$

$$\text{Constitutive relations of stresses: } \sigma = \bar{\rho}\frac{\partial \psi}{\partial \varepsilon^e} \ ; \ A = \bar{\rho}\frac{\partial \psi}{\partial \alpha} \tag{44}$$

$$\text{Yield function: } F(\sigma, A) \tag{45}$$

$$\text{Evolution of internal variables: } \dot{\varepsilon}^p = \dot{\gamma}N \ ; \ \dot{\alpha} = \dot{\gamma}H \tag{46}$$

$$\text{Loading/unloading conditions: } F \leq 0 \ ; \ \dot{\gamma} \geq 0 \ ; \ \dot{\gamma}F = 0 \tag{47}$$

5.2 FreeFEM Implementation

5.2.1 Physical Model

We concern ourselves with the model of a 50um by 75um block of copper impacting a rigid wall at 500m/s.

We proceed with von Mises' J_2 based associative flow with Johnson-Cook model. [**?**]

The yield function of the plasticity model is given by the difference of the von Mises stress and the Johnson-Cook yield stress.

$$F(q) = \sigma_{vm} - \sigma_Y \tag{48}$$

$$\sigma_{vm} = \sqrt{\frac{3}{2}s : s} \ ; \ s = \sigma - \frac{1}{3}\operatorname{Tr}(\sigma)I \tag{49}$$

$$\sigma_Y = (A + B\bar{\varepsilon}_p^{n_{JC}})(1 + C\ln(\dot{\epsilon}_p/\dot{\epsilon}_0))(1 - T^{*m}) \ ; \ T^* = \frac{T - T_0}{T_m - T_0} \tag{50}$$

$$\bar{\varepsilon}_p = \int |\dot{\varepsilon}_p| dt = \int \dot{\bar{\varepsilon}}_p dt \ ; \ \dot{\bar{\varepsilon}}_p = \sqrt{\frac{2}{3}\dot{\varepsilon}_p : \dot{\varepsilon}_p} \tag{51}$$

Where σ_{vm} is the von Mises stress, σ_Y the yield stress, $\bar{\varepsilon}_p$ is the equivalent/accumulated plastic strain.

For associative flow, the flow potential is equivalent to the yield function.

For the accumulation of plastic deformation and internal state variables, it will be given by the associative flow rule.

$$\dot{\varepsilon}_p = \dot{\gamma}\frac{\partial F}{\partial \sigma} = \dot{\gamma}\frac{3s}{2\sigma_{vm}} \tag{52}$$

$$\dot{\varepsilon} = \dot{\gamma}\frac{\partial F}{\partial A} = \dot{\gamma} \tag{53}$$

Fortunately for us, with von Mises' J_2 based Johnson-Cook associated flow plasticity, $\dot{\gamma}$ is equivalent to $\dot{\bar{\epsilon}}_p$.[**?**] This gives us a lot less to worry about in the return mapping algorithm later.

$$\dot{\bar{\varepsilon}}_p = \sqrt{\frac{2}{3}\dot{\varepsilon}_p : \dot{\varepsilon}_p} = \sqrt{\dot{\gamma}^2\frac{3s : s}{2\sigma_{vm}^2}} = \dot{\gamma} \tag{54}$$

Where we use the loading/unloading criterion to drop absolute value.

The power delivered via the plastic process is given by the following equation.

$$\dot{W}_p = \sigma : \dot{\epsilon}_p \tag{55}$$

We suppose only a fraction of the energy is turned into heat, so the temperature change is given by the following.

$$\dot{T} = \chi \frac{s : \dot{\epsilon}_p}{\rho C_p} = \chi \dot{\bar{\epsilon}}_p \frac{\sigma_{vm}}{\rho C_p} = T_k \dot{\bar{\epsilon}}_p \sigma_{vm} \tag{56}$$

χ is Taylor-Quincy heat fraction. Where we define $T_k = \frac{\chi}{\rho C_p}$ and call it the temp fraction.

We lay out all the constants used and move towards the numerical implementation.

```
// Copper particle
real rho        = 8960.;        // Density
real E          = 117e9;        // Young's modulus
real nu         = 0.34;         // Poisson modulus
real A          = 90e6;         // 1st Johnson-Cook
    hardening parameter
real B          = 292e6;        // 2nd Johnson-Cook
    hardening parameter
real nJC        = 0.31;         // Johnson-Cook hardening
    strain exponent
real C          = 0.025;        // Johnson-Cook rate
    parameter
real m          = 1.09;         // Johnson-Cook thermal
    exponent
real dEps0      = 1.0;          // Reference strain rate

real Tm         = 1356.;        // Melting temp
real Cp         = 385.;         // Specific heat
real chi        = 0.9;          // Heat fraction
real Tk         = chi/(rho*Cp); // Temp fraction
real T0    = 300.;              // Starting temp

real vimp  = -500.;     // Impact velocity
```

5.2.2 Elastic Portion only

We set up the model as a linear elastic problem, integrated over time to 50ns using Newark beta.

To avoid the cost of a contact algorithm, we idealize the circumstances and initiate the model right when the block hits the wall, and enforce a Dirichlet condition such that it doesn't penetrate it.

5.2.3 Plasticity with Linear Elasticity

We will adopt the backward euler return mapping as detailed in [**?**] as a special case of the generalized trapezoidal return mapping with $\theta = 1$.

We present the incremental form of the system of equations.

$$\varepsilon^e_{n+1} = \varepsilon^{e\ trial}_{n+1} - \Delta\gamma[(1-\theta)N_n + \theta N_{n+1}] \tag{57}$$

$$\alpha_{n+1} = \alpha_n + \Delta\gamma[(1-\theta)H_n + \theta H_{n+1}] \tag{58}$$

$$F(\sigma_{n+1}, A_{n+1}) = 0 \tag{59}$$

$$\varepsilon^p_{n+1} = \varepsilon^{p\ trial}_{n+1} + \Delta\gamma[(1-\theta)N_n + \theta N_{n+1}] \tag{60}$$

$$\sigma_{n+1} = \sigma^{trial}_{n+1} - \Delta\gamma D^e : [(1-\theta)N_n + \theta N_{n+1}] \tag{61}$$

$$\sigma = D^e : \varepsilon^e \tag{62}$$

Where D^e is the elasticity tensor. θ is a parameter to tune the return mapping for desired performances. With $\theta = 1$ we obtain 1st order accuracy and unconditional convergence with the von Mises J_2 type yield surface.

The elastic predictor/return mapping algorithm steps are as follows.

- 1. Elastic trial step: given $\Delta\varepsilon$ and state variables at current time predict the state at the next time step.

$$\varepsilon^{e\ trial}_{n+1} = \varepsilon^e_n + \Delta\varepsilon$$

$$\alpha^{trial}_{n+1} = \alpha_n$$

$$\sigma_{n+1} = \bar{\rho}\frac{\partial F}{\partial \varepsilon^e}\ ;\ A_{n+1} = \bar{\rho}\frac{\partial F}{\partial \alpha}$$

- 2. Check plastic admissibility.

$$\text{If } F(\sigma^{trial}_{n+1}, A^{trial}_{n+1}) \leq 0$$

$$\text{Update by setting } (\cdot)_{n+1} = (\cdot)^{trial}_{n+1} \text{ and restart loop.}$$

- 3. Return mapping algorithm. Solve the system.

$$\begin{bmatrix} \varepsilon^e_{n+1} - \varepsilon^{e\ trial}_{n+1} + \Delta\gamma[(1-\theta)N_n + \theta N_{n+1}] \\ \alpha_{n+1} - \alpha_n - \Delta\gamma[(1-\theta)H_n + \theta H_{n+1}] \\ F(\sigma_{n+1}, A_{n+1}) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

- 4. Restart loop

Or simplified in plain english: 1. assume time step is fully elastic. 2. Check that it actually is elastic and accept if so. 3. If plastic, correct the behavior to match plastic constraints. 4. Loop to next time step.

We now explicitly write out all the incremental formulas in the return mapping algorithm.

$$\begin{aligned}
\varepsilon_{n+1}^{p\ trial} &= \varepsilon_n^p \\
\varepsilon_{n+1}^{e\ trial} &= \varepsilon_n^e + \Delta\varepsilon = \varepsilon_{n+1} - \varepsilon_n^p \\
\bar{\varepsilon}_{n+1}^{p\ trial} &= \bar{\varepsilon}_n^{p\ trial} \\
\sigma_{Yn+1}^{trial} &= \sigma_{Yn} \\
s_{n+1}^{trial} &= 2\mu\varepsilon_{d\ n+1}^{e\ trial} \\
\varepsilon_{n+1}^{e} &= \varepsilon_{n+1}^{e\ trial} - \Delta\gamma\frac{3s_{n+1}}{2\sigma_{n+1}^{VM}} \\
\bar{\varepsilon}_{n+1}^{p} &= \bar{\varepsilon}_n^p + \Delta\gamma \\
s_{n+1} &= s_{n+1}^{trial} - 2\mu\Delta\gamma\frac{3s_{n+1}}{2\sigma_{n+1}^{VM}} \\
s_{n+1} &= \left(1 - \Delta\gamma\frac{3\mu}{\sigma_{n+1}^{VM\ trial}}\right) s_{n+1}^{trial} \\
\tilde{F}(\Delta\gamma) &= \sigma_{n+1}^{VM\ trial} - 3\mu\Delta\gamma - \tilde{\sigma}_Y(\Delta\gamma)
\end{aligned}$$

We now detail the numerical implementation.

The first step is to integrate an elastic trial step. We use Newark beta to integrate for 1 time step where we swapped the linear forcing term Kx with the following.

$$Kx \Longrightarrow D^e : (\Delta\varepsilon_n^e) - \boldsymbol{\nabla} \cdot \sigma_n = D^e : (\varepsilon(x_{n+1} - x_n)) - \boldsymbol{\nabla} \cdot \sigma_n$$

As shown in this block of code.

```
// Newark
problem Newark([xx,yy],[vi,vj])
        = int2d(Ph)(
                    c0(dt)*rho*M([xx,yy])+
                    eps([vi,vj])'*Elas*eps([xx,yy])
                    )
        + int2d(Ph)(eps([vi,vj])'*[SigXX,SigYY,SigXY])
        - int2d(Ph)(
                    c0(dt)*rho*M([xxo,yyo])+
                    c1(dt)*rho*M([vxo,vyo])+
                    c2(dt)*rho*M([axo,ayo])+
                    eps([vi,vj])'*Elas*eps([xxo,yyo])
                    )
        + on(left, xx=0);
```

We directly save the new kinematic quantities as they do not get changed in the return mapping. Just parts of the new displacement is delegated to elastic deformation or plastic deformation.

We setup all necessary trial information.

```
ElEpsXXt= eps([xx,yy])[0]-PlEpsXX;
ElEpsYYt= eps([xx,yy])[1]-PlEpsYY;
ElEpsZZt= 0-PlEpsZZ;
ElEpsXYt= eps([xx,yy])[2]-PlEpsXY;
Trace   = ThreeTrace([ElEpsXXt,ElEpsYYt,ElEpsZZt,ElEpsXYt]);
p       = (lambda+2/3*mu)*Trace; // it has sum of 2muSig_ii
    in hydrostatic
SigXXt  = lambda*Trace+2*mu*ElEpsXXt;
SigYYt  = lambda*Trace+2*mu*ElEpsYYt;
SigZZt  = lambda*Trace+2*mu*ElEpsZZt;
SigXYt  = mu*ElEpsXYt;
DevXXt  = Dev(p,[SigXXt,SigYYt,SigZZt,SigXYt])[0];
DevYYt  = Dev(p,[SigXXt,SigYYt,SigZZt,SigXYt])[1];
DevZZt  = Dev(p,[SigXXt,SigYYt,SigZZt,SigXYt])[2];
DevXYt  = Dev(p,[SigXXt,SigYYt,SigZZt,SigXYt])[3];
SigVMt  = SigmaVM([DevXXt,DevYYt,DevZZt,DevXYt]);
```

We check every single finite element to see if the flow criterion is satisfied on each, and move into the "newton-Raphson" solve.

```
for (int i = 0; i < Sh.ndof; i++){
  real DelG=0;

  if (YieldFunc(SigVMt[][i],EqPlEps[][i],0,Temp[][i])>
      YieldFuncTol){
  DelG = Newton(SigVMt[][i], EqPlEps[][i], Temp[][i]);
}
```

Whilst symbolic manipulation softwares exists, I no longer have access to them from my former institute, so we adopt a hybrid between secant and Newton without the need of an explicit derivative formula. We simply take the old input point and add a small perturbation to it to generate the derivative to the residue function $\tilde{F}$.

$$x_{n+1} = x_n - f(x_n)\frac{h}{f(x_n+h) - f(x_n)}$$

Where we also add in some protection against dividn by something close to 0. Given by the following code block.

```
// Newton-Raphson. Well more of a secant
func real Newton(real vmTrial, real epsEqOld, real Told)
{
  real dg = 0;
  for (int it = 0; it < newtiters; it++){
        real epsRate = dg/dt;
        real Tguess  = Told + TempGen(epsEqOld+dg, dg, Told,
             dt);
        real sY      = SigmaY(epsEqOld+dg, epsRate, Tguess);
        real R       = vmTrial - 3.*mu*dg - sY;
        if (abs(R) < 1.0e-6*newttol) break;
```

```
        real h        = max(dg*1.0e-6, 1.0e-12);
        real epsRateh = (dg+h)/dt;
        real Tguessh  = Told + TempGen(epsEqOld+dg+h, dg+h,
            Told, dt);
        real sYh      = SigmaY(epsEqOld+dg+h, epsRateh,
            Tguessh);
        real Rh       = vmTrial - 3.*mu*(dg+h) - sYh;

        real dRdg = (Rh-R)/h;
        if (abs(dRdg) < 1.0e-20) break;

        real dgNew = dg - R/dRdg;
        if (dgNew < 0.0) dgNew = 0.0;
        dg = dgNew;
        if(it==newtiters) cout << "Newton iteration limit
            reached" << endl;
  }

  return dg;
}
```

After the Newton iteration converges, we update the remaining field variables using the incremental formulas from prior.

```
EqPlEps[][i] += DelG;
DevXX[][i]       = (1.- 3.*mu*DelG/SigVMt[][i])*DevXXt[][i];
real[int] sn = [DevXX[][i], DevYY[][i], DevZZ[][i], DevXY[][
    i]];
real factor = 1.5*DelG/SigVMt[][i];
PlEpsXX[][i] += factor*sn[0];
PlEpsYY[][i] += factor*sn[1];
PlEpsZZ[][i] += factor*sn[2];
PlEpsXY[][i] += factor*sn[3];
//  Is it still heating up too much? Is this line also
    causing troubles?
Temp[][i]        += TempGen(EqPlEps[][i], DelG, Temp[][i],
    dt);
SigXX[][i]       = p[][i] + DevXX[][i];
SigYY[][i]       = p[][i] + DevYY[][i];
SigZZ[][i]       = p[][i] + DevZZ[][i];
SigXY[][i]       = DevXY[][i];
```

5.2.4 Results

The results can be viewed as a time series in Paraview, or any .vtu viewer.

We first present the elastic results. At the end of those 50ns, we already see the block begin to rebound off the wall. We are definitely accumulating errors due to not using a contact algorithm.

In the plastic case, we see that the material deforms significantly more, and ends with sticking to the wall with the interface at 150 degrees higher.

6 Cold Spray

6.1 Overview

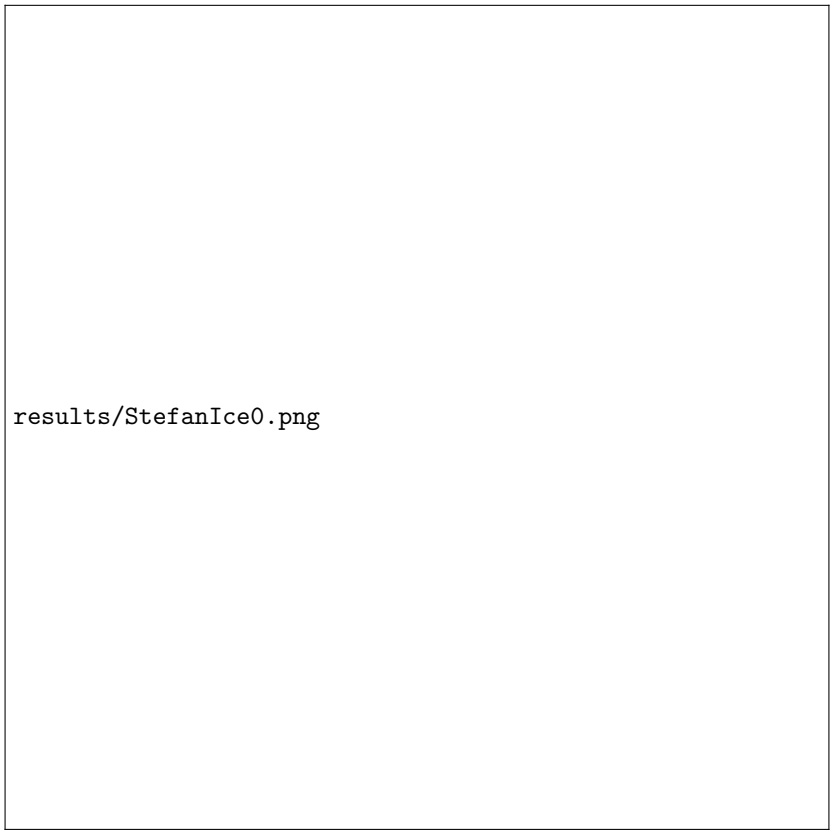
6.2 FreeFEM Implementation

6.2.1

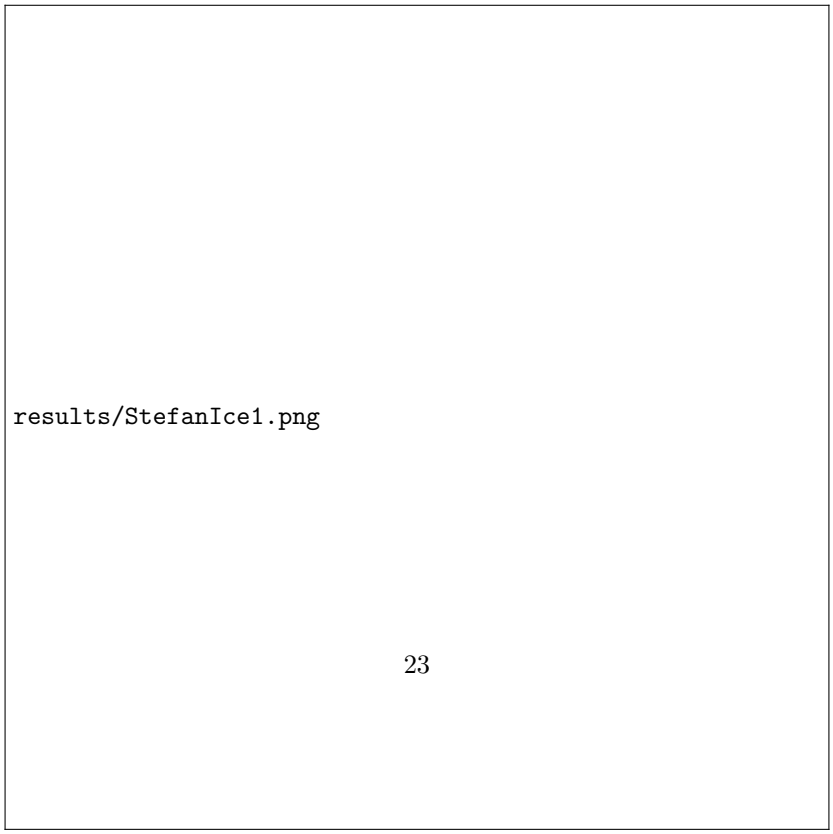
We must now combine all 3 other subproblems into 1.

Fortunately, the algorithms studied so far easily combine. Recall that the contact algorithm first resolves the contact problem, then goes into the elasticity portion. The plasticity return mapping algorithm first integrate elastically, then resolves the plastic portion. This naturally tells us to simple swap the elastic portion in the contact algorithm with the elastoplastic version.

6.2.2 Results



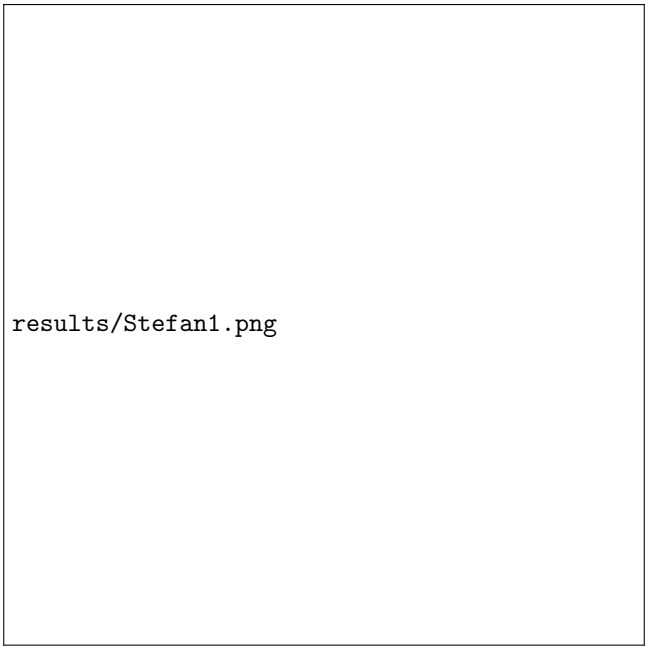
(a) Ice and water at the start



(b) Ice and water at the end

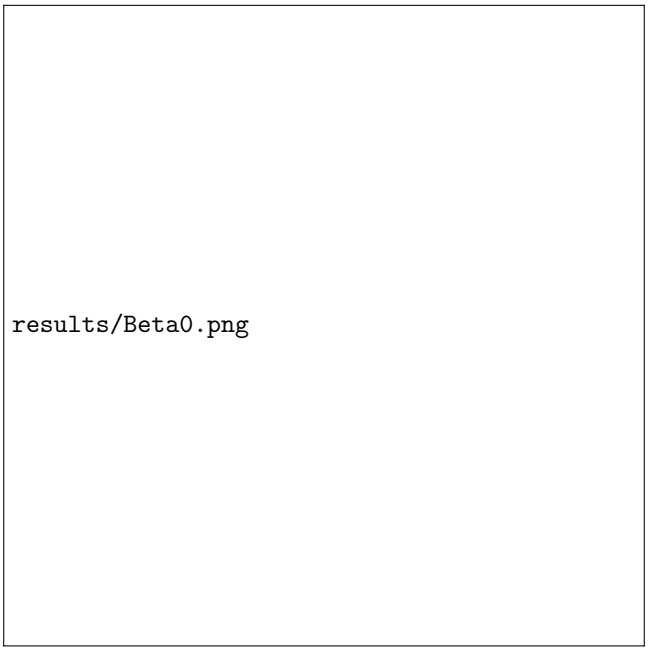

results/Stefan0.png

(a) t=0s

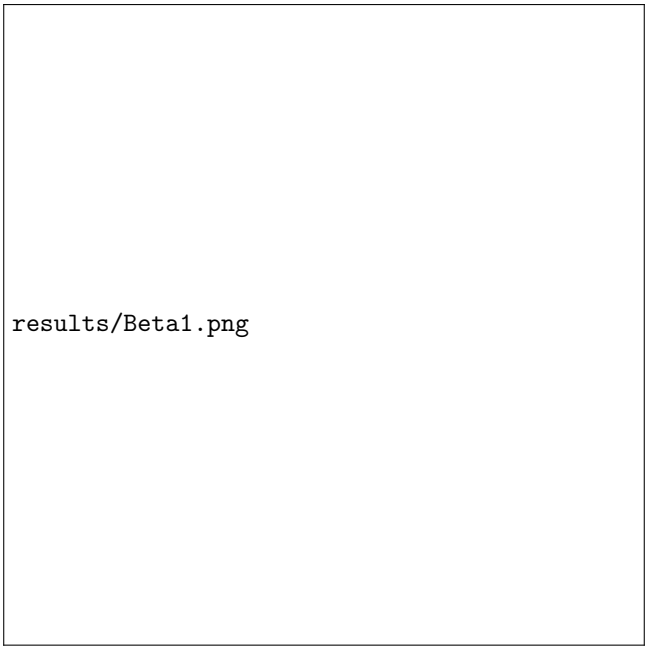

results/Stefan1.png

(b) t=7.5s

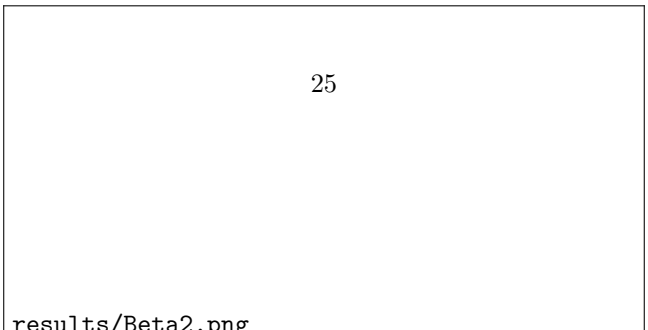

results/Stefan2.png



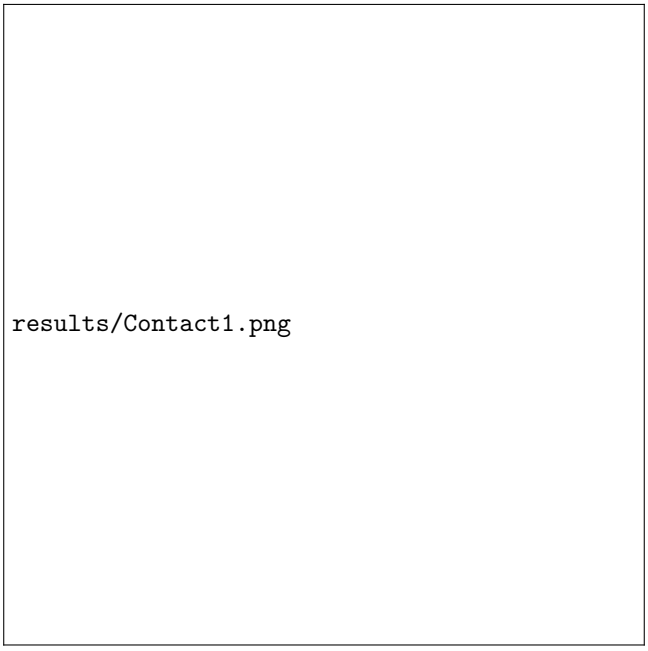
(a) t=0s

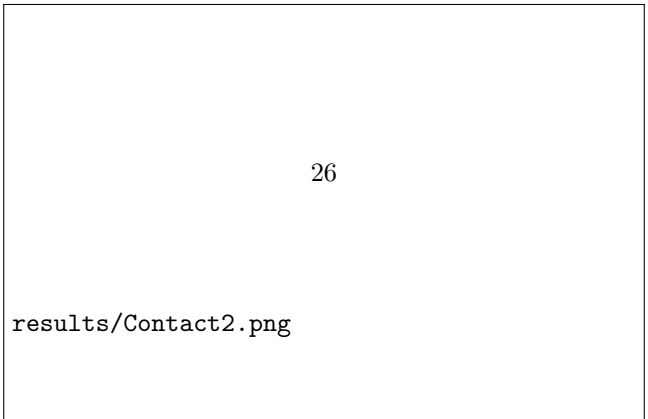


(b) t=0.75s

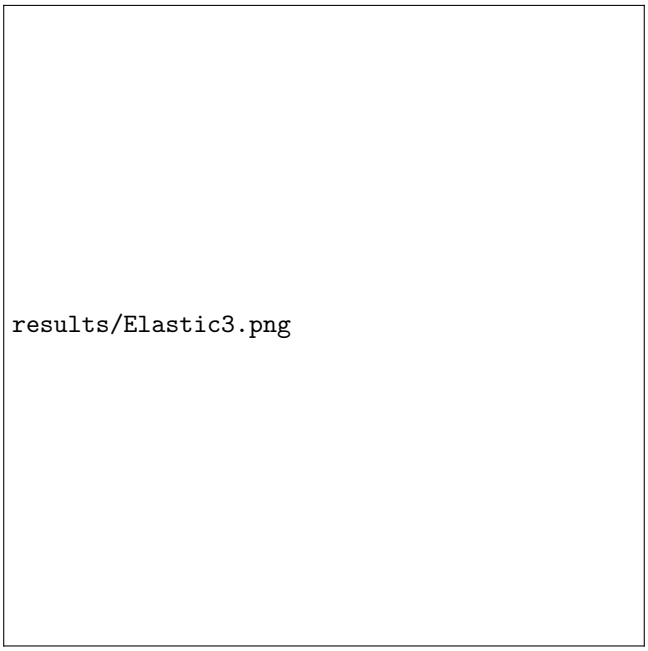



results/Contact0.png


results/Contact1.png

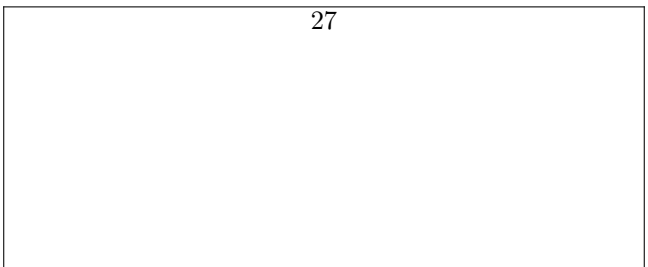

results/Contact2.png

Elastic Deformation only and Pressure Waves

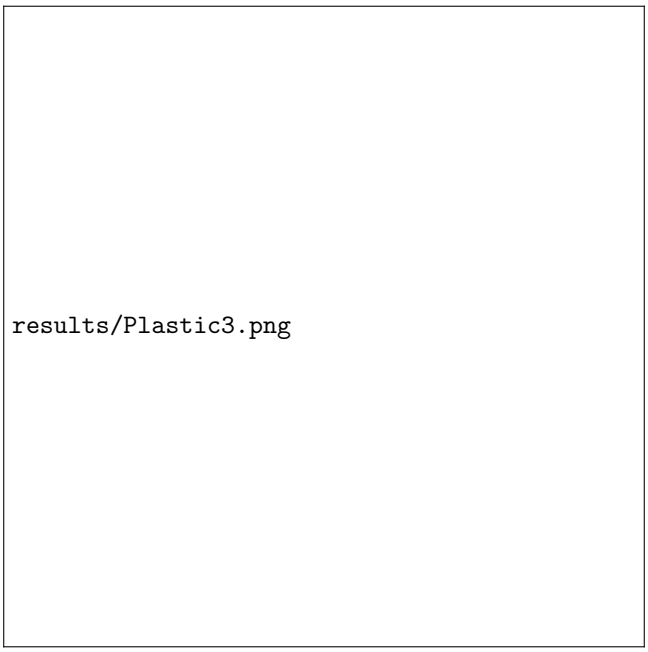


(a) t=4.2ns

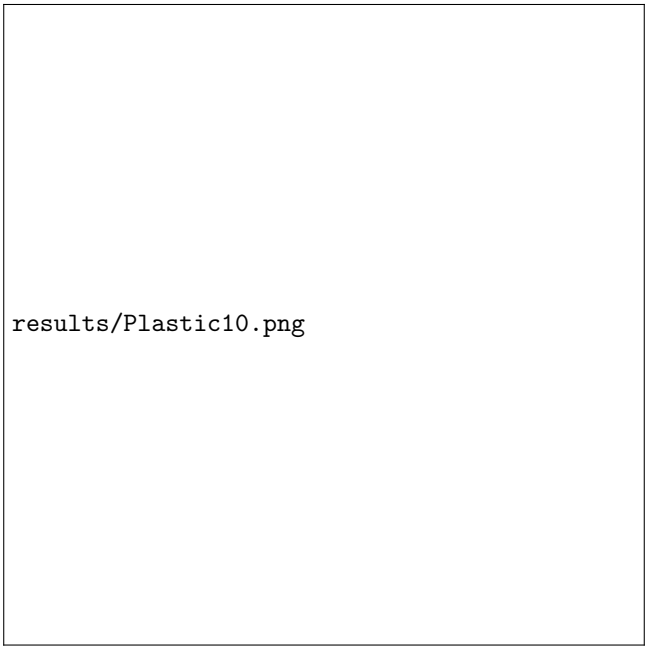
(b) t=14s



Plastic Deformation and Pressure Waves



(a) t=4.2ns



(b) t=14s